

1 Типовые операции нейронных сетей на GPU

Теперь рассмотрим типовые операции нейронных сетей с точки зрения вычисления их на GPU, ведь разные операции по-разному ложатся на архитектуру GPU и, как мы поняли в предыдущем разделе, требуют разных оптимизаций.

1.1 Поэлементные операции

Самый простой тип операции — поэлементные операции. Например, такой операцией является сложение двух тензоров, как при использовании residual connections:

$$c_i = a_i + b_i$$

или же поэлементно применяются некоторые функции активации, например ReLU:

$$y_i = \max(0, x_i)$$

причем в данном примере $\max$ не является редукцией в полном смысле слова, поскольку вычисляется всего над двумя элементами.

В поэлементных операциях каждый элемент результата зависит только от одного или нескольких элементов входных тензоров с тем же индексом.

Такие операции очень легко распараллеливаются, но в то же время обычно обладают низкой арифметической интенсивностью, что делает их memory-bound операциями.

1.2 Редукции

В нейронных сетях часто встречаются редукции — операции, которые преобразуют массив элементов в скаляр.

В отличие от поэлементных операций, редукции параллелизуются сложнее. В наивном случае, это приведет к тому, что несколько потоков будут пытаться писать значения в одну ячейку памяти. Поэтому редукции требуют дополнительных синхронизаций.

1.3 GEMV

GEMV-операция — это умножение вектора на матрицу. Например преобразование одного вектора с помощью линейного слоя:

$$y = xW$$

Подобная операция часто встречается при инференсе трансформеров с использованием KV-кэша (что мы обсудим позже).

С одной стороны, GEMV хорошо распараллеливается по столбцам матрицы. С другой стороны, GEMV имеет значительно меньшую арифметическую интенсивность, чем GEMM.

1.4 GEMM

GEMM — это операция матричного умножения, самая важная операция для нейронных сетей:

$$Y = XW$$

Каждый элемент матрицы Y вычисляется независимо друг от друга, а значит параллельно. Таким образом, один поток программы может вычислять один элемент матрицы Y . В наивном виде работа одного потока выглядит так:

$$Y_{ij} = \sum_{k=1}^D X_{ik} W_{kj}.$$

Такой подход имеет крайне низкую арифметическую интенсивность. Но важно то, что данные в GEMM можно переиспользовать.

Так, например, элемент X_{ik} будет также использоваться при вычислениях всех элементов i -ой строки матрицы Y , а элемент W_{kj} — при вычислении всех элементов j -го столбца матрицы Y . Эти элементы мы можем кэшировать в SMEM, значительно сокращая количество обращений к GMEM.

Чтобы в свою очередь сократить количество обращений к SMEM, мы можем вычислять одним потоком не 1 элемент матрицы, а, например, 2, 4, 8, или даже 128 элементов (при использовании FP4 или FP8 типов). Однако мы не можем наращивать это число до бесконечности, поскольку такой способ вычисления матриц колоссально увеличивает регистровое давление и снижает оверсапу.

Это позволяет значительно снизить задержку, вызванную чтением данных с GMEM. А чтобы увеличить вычислительную пропускную способность GEMM, используются тензорные ядра.

1.5 Нормализации

Нормализации используются почти во всех современных архитектурах.

Рассмотрим особенности вычисления нормализаций на примере LayerNorm:

$$y_j = \gamma_j \frac{x_j - \mu}{\sqrt{\sigma^2 + \varepsilon}} + \beta_j,$$

где

$$\mu = \frac{1}{D} \sum_{j=1}^D x_j$$

и

$$\sigma^2 = \frac{1}{D} \sum_{j=1}^D (x_j - \mu)^2$$

Нормализация выполняется по одному токёну. Для каждого токёна нам необходимо выполнить редукции: посчитать среднее и сумму квадратов. Самой арифметики в нормализации немного, что делает нормализацию — memory-bound операцией.

Однако, нормализация сложнее обычных поэлементных операций, потому что редукции также добавляют дополнительные синхронизации.

2 Программная модель CUDA

Перейдем к инструментам программирования на GPU. Начнем с основы работы с GPU NVIDIA — CUDA.

2.1 Технологический стек CUDA

CUDA — это целый стек технологий для работы с GPU.

На нижнем уровне находится **драйвер**. Он отвечает за взаимодействие с устройством: управление GPU, выделение памяти, запуск программ и работу с низкоуровневыми ресурсами.

Над драйвером находится **CUDA Runtime** — набор библиотек, инструментов и API. CUDA Runtime предоставляет более удобный интерфейс для работы по сравнению с драйвером. Так, например, есть функции для выделения памяти и её копирования между хостом и девайсом.

Над CUDA Runtime находится **CUDA C/C++** — расширение языка C++, которое позволяет писать программы, работающие с CUDA Runtime. Такие программы компилируются с помощью специального компилятора — **NVCC** (NVIDIA CUDA Compiler).

Еще выше находятся специализированные библиотеки, такие как cuBLAS и cuBLASLt для операций линейной алгебры, cuDNN с примитивами для нейронных сетей, NCCL для коммуникации между несколькими девайсами и многие другие. Из всех них мы в дальнейшем разберем только NCCL.

2.2 CUDA-кernels

Функция, выполняемая на девайсе, называется **кэрнел**. Кэрнел описывает работу одного потока. При запуске таких потоков создаётся много, поэтому каждый поток получает свой индекс и обрабатывает свой элемент массива.

Программа, которую мы пишем, обрабатывается на хосте. Хост также инициализирует запуск кэрнелов. Запуск кэрнела — не бесплатный. Хост должен отправить работу на GPU, подготовить параметры запуска и поставить операцию в очередь выполнения. Все это приводит к **накладным расходам** — **kernel launch overhead**.

Если кэрнел большой и выполняет много работы, то эти накладные расходы будут несущественны. Но если программа состоит из большого числа маленьких кэрнелов, то kernel launch overhead становится существенным.

2.3 Потоки, блоки, сетка

Вычисления в CUDA организованы иерархически:

Grid $\rightarrow$ Blocks $\rightarrow$ Threads

Поток (thread), как мы ранее обсуждали, это логическая единица выполнения программы, не аппаратная. Каждый поток в CUDA работает в рамках исполнительской модели SIMT: он исполняет один и тот же код ядра, но работает со своим индексом и со своими данными.

Потоки объединяются в варпы по 32 штуки, однако такой ступени иерархии в CUDA явно нет (хотя на аппаратном уровне вычисления все равно происходят именно варпами).

Зато есть **блоки (blocks)** — тоже группы потоков. В отличие от варпов, блоки не имеют фиксированный размер, он задается программистом (но обычно он кратен 32, поскольку иначе будет вызван варп, некоторые потоки которого будут отключены). Потоки внутри блока могут взаимодействовать друг с другом: обращаться к общей SMEM и синхронизироваться. В одном CUDA-блоке может максимум 1024 потока (то есть 32 варпа).

Набор блоков, которые запускаются при вызове ядра, называется **сетка (grid)**.

2.4 Индексация

Каждый поток имеет свой индекс внутри блока, каждый блок имеет свой индекс внутри сетки. Чтобы определить глобальный индекс потока, в ядре также доступны размер блока и размер сетки.

Поскольку часто CUDA-ядрам необходимо работать с многомерными массивами, для упрощения работы используется индексация потоков внутри блока и блоков внутри сетки по трём осям: x , y и z .

2.5 Синхронизации

Потоки асинхронны и могут запускаться в любом порядке. Если программа построена таким образом, что какой-то поток зависит от другого, то необходимо убедиться, что зависимый поток продолжит свое исполнение только после того, как основной поток пройдет некоторый этап программы.

В CUDA есть несколько синхронизаций: синхронизация на уровне девайса, синхронизация на уровне ядра и синхронизация на уровне варпа.

Синхронизации стоит рассматривать как барьер, который ждет завершения выполнения всех участников группы, чтобы продолжить код.

В первом случае синхронизация проверяет, что все ядра на всех стримах выполнены и безопасно начинать новую задачу.

Во втором случае синхронизация ставит барьер для работы потоков внутри ядра. Это полезно, если работа на нескольких потоках идет с одним и тем же участком памяти и требуется “выравнивание” работы всех

потоков, прежде чем вносить изменения, чтобы избежать такой проблемы, как race conditions.

Третий случай аналогичен второму, но синхронизация происходит не между всеми потоками, а между потоками одного варпа.

2.6 Race Conditions

Race conditions (гонки) — ситуация, когда результат зависит от недетерминированного порядка выполнения потоков, потому что несколько потоков без должной синхронизации читают или записывают в одни и те же ячейки памяти. Грубо говоря, разные потоки исполняются наперегонки, без необходимой синхронизации своих инструкций.

Чаще всего race conditions вызваны гонками данных, когда два потока пытаются изменить данные по одному адресу и между ними нет никакого барьера.

Другая ситуация — один поток должен прочесть результат, а второй поток этот результат еще не записал в нужную ячейку памяти.

Гонки опасны тем, что приводят к некорректному результату вычислений без явного падения программы.

Ранее мы говорили о конфликтах банков в SMEM. Конфликты банков не являются гонками. В отличие от гонок, конфликты банков приводят лишь к потере производительности.

2.7 Редукции

Редукции требуют более сложной организации параллелизма, чем обычные вычисления. Рассмотрим на примере суммы массива:

$$s = \sum_{i=1}^N x_i$$

При наивном подходе, все потоки одновременно добавляют свои значения в одну переменную, что приводит к гонкам и некорректному результату.

Воспользуемся ассоциативностью сложения, будем вычислять редукцию по дереву.

Разделим с округлением вверх количество элементов массива на размер блока. Полученное значение — это количество элементов массива, за которые ответственен 1 поток. Найдем частичные суммы s_i для каждого потока, последовательно сложив все элементы.

Внутри одного варпа мы можем разбить потоки (и соответствующие элементы) на пары.

$$\underbrace{s_0, s_1}_{\text{Пара 1}}, \underbrace{s_2, s_3}_{\text{Пара 2}}, \dots, \underbrace{s_{28}, s_{29}}_{\text{Пара 15}}, \underbrace{s_{30}, s_{31}}_{\text{Пара 16}}$$

Поток с меньшим индексом прибавит к своему элементу элемент соседа. После чего останется 16 частичных сумм:

$$\underbrace{s_0 + s_1}_{z_0}, \underbrace{s_2 + s_3}_{z_1}, \dots, \underbrace{s_{28} + s_{29}}_{z_{14}}, \underbrace{s_{30} + s_{31}}_{z_{15}}$$

На следующем шаге суммы снова объединяются попарно:

$$\underbrace{z_0, z_1}_{\text{Пара 1}}, \underbrace{z_2, z_3}_{\text{Пара 2}}, \dots, \underbrace{z_{12}, z_{13}}_{\text{Пара 7}}, \underbrace{z_{14}, z_{15}}_{\text{Пара 8}},$$

Повторим эти шаги в сумме 5 раз (поскольку $\log_2 32 = 5$) и в первом потоке варпа окажется сумма по всем потокам варпа. Между каждыми такими шагами необходимо синхронизировать потоки внутри варпа, чтобы избежать гонок.

Эту частичную сумму сохраним в SMEM. Редукция внутри варпа по дереву называется **варп-редукцией**.

Поскольку в одном блоке может быть максимально 32 варпа, то в SMEM может быть сохранено максимально 32 частичные суммы. А это значит, что мы можем повторить варп-редукцию по частичным суммам в SMEM и получить итоговую сумму.

2.8 Стримы и асинхронность

Кернелы запускаются с хоста, но обычно выполняются на девайсе асинхронно относительно хоста, что означает, что CPU может поставить операцию на выполнение и продолжить работу, не дожидаясь немедленного завершения кернела на девайсе.

Для организации таких операций используются стримы (**streams**) — очереди операций на девайсе. Операции внутри стрима выполняются в порядке добавления.

Если операции находятся в одном стриме, то следующая операция начнётся только после завершения предыдущей. Однако разные стримы могут выполняться асинхронно относительно друг друга.

Это важно для обучения нейронных сетей. Например, на одном стриме производятся вычисления прямого и обратного этапов, а также обновляются параметры сети, на втором стриме находятся коммуникации между разными девайсами, третий стрим загружает на девайс следующий батч и так далее.

Причем, второй и третий стримы можно “спрятать” под первым стримом.

Однако, асинхронность усложняет общение между стримами и оценку времени работы кернела. Чтобы избежать ненужных синхронизаций в первом случае и корректно оценивать время работы различных участков CUDA-программы во втором случае, используются CUDA-события.

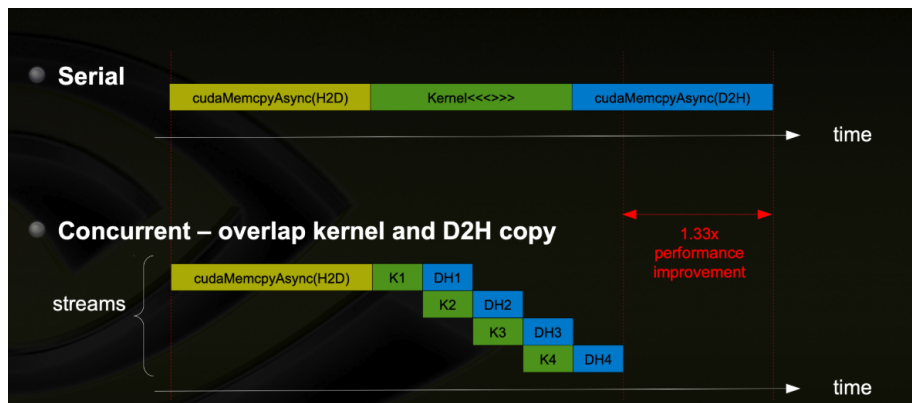


Рис. 1: На этом примере разделение выполнения ядра и копирования данных с девайса на хост на 4 стрима позволяет спрятать большую часть задержки выполнения копирования под выполнение ядра

2.9 CUDA-события

CUDA-события — это маркеры, которые можно записать в стрим. Когда девайс доходит до этого маркера, событие считается выполненным.

CUDA-события очень важны при работе с несколькими стримами. Например, при обучении нейронных сетей на нескольких девайсах.

Например, перед вычислением слоя, нам необходимо дождаться сборки необходимых данных с нескольких девайсов через ядро коммуникации. Ядра коммуникаций выполняются на отдельном стриме. Если стрим вычислений начнет вычисление слоя до того, как завершится стрим коммуникаций и с других девайсов будут собраны необходимые данные, то вычисления пойдут.

Это поведение напоминает race conditions. Гонки мы решали тем, что добавляли синхронизации. Добавим синхронизацию девайса.

Однако если ядро коммуникации завершится до того, как начнется ядро вычисления слоя, то стрим коммуникаций будет простаивать из-за синхронизации, хотя он мог бы начать выполнение следующей коммуникации (например, собирать данные для еще одного слоя).

CUDA-события позволяют нам поставить маркер после завершения ядра коммуникации и запретить выполнение ядра вычислений до того, как ивент на стриме коммуникаций будет выполнен.

Другой пример — замер времени выполнения ядра. Поскольку код, который замеряет время, исполняет CPU, а ядра запускаются асинхронно, то он будет замерять лишь время инициализации ядра (что в некотором смысле тоже может быть полезно, но это уже другая задача).

Мы можем поставить CUDA-события в стриме сразу перед ядром и сразу после него, а затем посчитать время между ними.